AI Business Copilot

Interview Preparation Guide

Architecture Decisions | Why These Choices | Pros & Trade-offs | Deep-Dive Q&A

1. PROJECT OVERVIEW

Q1. Tell me about your AI Business Copilot project. What does it do and what problem does it solve?

Answer: AI Business Copilot is a full-stack intelligent assistant that helps business users analyze their own documents (PDFs like sales reports, financial statements) simply by asking questions in natural language. It removes the need to manually read documents, run SQL queries, or build separate dashboards.

A user uploads their PDF documents, and can then ask things like: *"Summarize this report"*, *"Why did Q3 revenue drop?"*, or *"Show me a dashboard of our KPIs"*. The system intelligently routes each query to the right AI pipeline and returns a precise, contextually grounded answer.

Component	Technology	Purpose
Backend API	FastAPI + Uvicorn	Serves all endpoints, handles uploads, streams responses
AI Workflow	LangGraph StateGraph	Orchestrates 8 specialized agents in a multi-path graph
RAG Pipeline	FAISS + BM25 + HuggingFace	Retrieves relevant document chunks for each query
LLM	Groq API (gpt-oss-120b)	Powers all agent reasoning and text generation
Frontend	Next.js 16 + TypeScript	Chat UI, PDF upload, dynamic dashboard rendering
Auth & DB	Supabase	Google OAuth login, per-user data isolation
Deployment	Google Cloud Run + Vercel	Backend on GCP, frontend on Vercel
CI/CD	GitHub Actions	Auto-test + auto-deploy on every push to main

Q2. What are the 4 query types your system handles and how does routing work?

The Router Agent classifies every incoming query into one of 4 intents, then LangGraph routes it to the appropriate pipeline:

Intent	Triggered When	Agent Path	Example Query
analysis	Why/how questions needing deep investigation	Research → Analyst → Strategy → Response	"Why did Q3 revenue drop?"
lookup	Simple factual questions needing one answer	Research → Response	"What was our total revenue?"
summarize	Requests to condense/summarize documents	Research → Summary → Response	"Give me the key highlights"
dashboard	Requests for charts or visualizations	Research → DataExtractor → Dashboard → Response	"Show me a KPI dashboard"

Design Choice: Keyword-based routing is tried first (fast and cheap). LLM classification is only used as fallback for ambiguous queries. This reduces latency and cost for common patterns.

2. LANGGRAPH & MULTI-AGENT ARCHITECTURE

Q3. Why did you use LangGraph? Why not LangChain LCEL, CrewAI, or AutoGen?

Answer: LangGraph was the right choice because my use case required **conditional multi-path routing** — different queries need completely different agent pipelines. LangGraph lets me define this as an explicit directed graph with conditional edges, which none of the alternatives do cleanly.

Framework	What It's Good At	Why It Didn't Fit My Use Case
LangGraph (chosen)	Explicit graph control, conditional routing, shared state, deterministic flows	— Perfect fit for my 4-path architecture
LangChain LCEL	Linear chain of operations (A → B → C)	No conditional branching — can't route analysis vs lookup vs dashboard differently
CrewAI	Role-based agent collaboration where agents decide their own task order	Too autonomous — I need deterministic routing, not agents deciding their own path
AutoGen	Conversational multi-agent with back-and-forth dialogue	Designed for agent-to-agent chat, not structured business query pipelines
Custom code	Full flexibility	Would require reimplementing state management, graph execution, and error handling from scratch

□ PROS / WHY WE CHOSE THIS	□ CONS / TRADE-OFFS
Explicit graph structure — I can see exactly which agents run for each intent	Steeper learning curve than simple LCEL chains
Conditional edges make routing clean and testable	Overkill for simple single-path applications
Shared BusinessState TypedDict — all agents read/write the same state object	Graph must be rebuilt/recompiled on change (not hot-reloadable)
Graph compilation catches errors before runtime	
Built-in support for parallel node execution	
First-class LangChain integration	

Q4. Explain BusinessState — what is it and why did you use TypedDict?

Answer: BusinessState is a Python TypedDict that acts as the **shared memory** between all 8 agents. Every agent receives the full state, adds or updates its own fields, and returns the modified state.

I used **TypedDict** instead of a regular dict or Pydantic model because: (1) TypedDict gives IDE autocompletion and type checking without runtime overhead, (2) LangGraph requires the state to be a TypedDict or dataclass, and (3) it documents the contract between agents explicitly in code.

Fields: *query, user_id, intent, context, analysis, recommendations, summary, extracted_data, dashboard_code, final_response* — each agent only writes to the fields it's responsible for, all others flow through unchanged.

Interview Tip: When asked about state, explain the immutable update pattern: {**state, 'key': value} creates a new dict rather than mutating the existing one. This prevents side effects between agents.

Q5. How did you implement the router agent? Why use keyword matching before the LLM?

Answer: The router uses a two-tier approach — keyword matching first, LLM classification as fallback. This was a deliberate performance and cost optimization.

Most real-world queries are not ambiguous. A query containing 'dashboard', 'chart', or 'visualize' is **always** a dashboard request. Invoking an LLM to classify that is slow (~300ms extra) and wastes tokens. The keyword approach handles ~70% of queries in <1ms.

The LLM fallback handles genuinely ambiguous queries like *"Tell me about Q3"* that could be analysis, lookup, or summary.

PROS / WHY WE CHOSE THIS	CONS / TRADE-OFFS
Reduces average latency by ~200-300ms for common queries	Keyword lists need maintenance as product evolves
Reduces LLM cost — fewer API calls	Keyword matching can fail on unusual phrasing
Predictable — keyword routing never hallucinates	Adds code complexity over pure LLM routing
LLM fallback handles edge cases gracefully	

3. RAG PIPELINE DESIGN

Q6. Why did you choose RAG over fine-tuning the model on business data?

Answer: RAG was the correct choice for a business document assistant for 3 key reasons:

- Data freshness: Business documents change constantly. Fine-tuning would require retraining every time a new report is uploaded — impossible for a user-uploaded document system.
- Per-user isolation: Each user has their own documents. You can't fine-tune separate models per user. With RAG, I maintain separate FAISS indexes per user_id.
- Cost: Fine-tuning large models costs hundreds to thousands of dollars per run. RAG uses a pretrained embedding model (free to self-host) and a retriever.
- Verifiability: RAG responses are grounded in retrieved chunks — you can trace every answer back to a document passage. Fine-tuned models can hallucinate.

Approach	Best For	My Situation
RAG (chosen)	Dynamic, user-specific, frequently updated knowledge	Users upload new PDFs constantly — RAG updates in seconds
Full Fine-tuning	Changing model behavior/style/domain vocabulary	Doesn't help with new document knowledge
PEFT/LoRA	Efficient domain adaptation at lower cost	Still doesn't update with new documents — needs retraining
Prompt stuffing	Small documents that fit in context window	Sales reports can be 50+ pages — won't fit in context

Q7. Walk me through your RAG pipeline end-to-end — from PDF upload to response.

Ingestion (when user uploads PDFs):

- PyPDFLoader reads each page of the PDF into LangChain Document objects
- RecursiveCharacterTextSplitter splits into 500-char chunks with 50-char overlap
- BAAI/bge-small-en-v1.5 embedding model converts each chunk to a dense vector
- FAISS stores all vectors in a per-user index (faiss_indexes/{user_id}/)
- Chunks also saved to chunks.pkl (pickle) for BM25 retrieval

Retrieval (when user asks a question):

- Query expansion: LLM generates 3 query variations for broader coverage
- For each variation: FAISS similarity_search(k=4) + BM25 keyword search (k=4) = up to 32 raw docs
- Deduplication: hash first 100 chars of each chunk, remove duplicates
- Cross-encoder reranking: ms-marco-MiniLM-L-6-v2 scores every (query, chunk) pair
- Top 6 chunks returned as context string to the appropriate agent

Q8. Why did you choose chunk_size=500 and chunk_overlap=50? How did you decide these values?

Answer: These values are based on a trade-off between context richness and retrieval precision:

Parameter	Value Chosen	If Too Small	If Too Large
chunk_size	500 chars (~100 words)	Chunks lose surrounding context, answers are incomplete	Chunks are too broad, retrieval returns irrelevant content
chunk_overlap	50 chars (~10 words)	Information at chunk boundaries gets cut off	Too much redundancy, wastes index space

500 characters is roughly **1-2 sentences to a short paragraph** — enough to contain a complete business fact (e.g. 'Q3 revenue was $2.9M, down 8% from Q2') without including unrelated content from the surrounding page. Business documents tend to have self-contained data points at this granularity.

Q9. Why FAISS for vector storage? Why not Pinecone, Weaviate, ChromaDB, or Qdrant?

Vector Store	Type	Why Not Chosen for This Project
FAISS (chosen)	Local, in-memory/on-disk	Free, no network latency, works offline, persists to disk, no per-query cost
Pinecone	Managed cloud	Paid service ($70+/month), adds network latency, overkill for document sizes here
ChromaDB	Local embedded	Simple API but slower at scale, less battle-tested than FAISS
Weaviate	Self-hosted or cloud	Heavy infrastructure — requires running a separate service/container
Qdrant	Self-hosted or cloud	Great for production scale, but adds ops overhead for this project size

Key reasons for FAISS: (1) The document sizes in this project (10-100 page PDFs) result in 200-2000 chunks — well within FAISS's sweet spot. (2) faiss-cpu runs in the same Python process, so retrieval is microseconds, not milliseconds. (3) No external service means one less moving part in deployment. (4) Zero ongoing cost.

When to switch: If document volume scaled to millions of chunks, or if multiple backend instances needed to share the same index, I would migrate to Qdrant (self-hosted) or Pinecone (managed cloud) — both support distributed, multi-tenant indexing.

4. ADVANCED RAG TECHNIQUES

Q10. What is Query Expansion and why did you add it to your RAG pipeline?

Answer: Query expansion solves a fundamental problem in RAG: **vocabulary mismatch**. The user's query and the document may use different words for the same concept.

Example: User asks *"What was our profit?"*. The document says *"Net income was $420K"*. A simple embedding similarity search may not retrieve this if 'profit' and 'net income' sit in different vector neighborhoods.

Solution: Generate 3 LLM-created variations of the query (e.g., *"net income"*, *"earnings"*, *"bottom line result"*) and run retrieval on all of them. The union of results covers the vocabulary gap.

□ PROS / WHY WE CHOSE THIS	□ CONS / TRADE-OFFS
Significantly improves recall — especially for business jargon	Adds one LLM call per query (~100ms extra latency)
Catches synonym gaps without retraining the embedding model	LLM may occasionally generate poor query variations
Low implementation cost — one LLM call before retrieval	Multiplies retrieval calls (3x queries = 3x FAISS + BM25 calls)
Pairs well with deduplication to avoid duplicate results	

Q11. Why did you use Hybrid Search (BM25 + FAISS)? Why not just semantic search?

Answer: Semantic search (FAISS) and keyword search (BM25) have complementary strengths. Using only one leaves gaps.

Search Type	Strengths	Weaknesses

FAISS (semantic)	Finds conceptually similar content even with different words; understands meaning	Can miss exact matches for specific numbers, names, or codes
BM25 (keyword)	Excellent at exact term matching — great for numbers, product codes, names	Fails when query and doc use different vocabulary
Hybrid (chosen)	Best of both worlds — semantic understanding + exact match precision	More complex, ~2x retrieval calls per query

Example: Query *"What was SKU-2042 sales in Q3?"*. BM25 will find 'SKU-2042' by exact match. FAISS might drift to semantically similar product-related passages. Combining both gives the right answer with confidence.

Q12. What is Reranking and why did you use a cross-encoder? Why not just take the top-k from FAISS?

Answer: After hybrid search, we may have 20-30 candidate chunks. FAISS scores them by **approximate nearest-neighbour distance** — which measures how close the query and chunk embeddings are in vector space. This is a proxy for relevance, not true relevance.

A **cross-encoder** takes **both** the query and a passage as input together and produces a single relevance score. This is much more accurate because the model can see the full context of both, including exact word matches and logical relationships.

Model Type	How It Works	Speed	Accuracy
Bi-encoder (FAISS embedding)	Encodes query and doc separately, compares vectors	Fast — pre-computed doc embeddings	Good — misses fine-grained relevance
Cross-encoder (reranker)	Encodes (query, doc) pair together	Slow — must run on every (query, doc) pair	Excellent — sees both inputs jointly

Why **ms-marco-MiniLM-L-6-v2** specifically: It was trained on MS MARCO, a large dataset of (query, passage) relevance pairs from real search scenarios. MiniLM-L-6-v2 is a 22M parameter distilled model — fast enough for real-time use (~50ms for 20 chunks) while maintaining strong accuracy.

Q13. Why BAAI/bge-small-en-v1.5 for embeddings? Why not OpenAI text-embedding-ada-002?

Embedding Model	Cost	Privacy	Latency	Quality
BAAI/bge-small-en-v1.5 (chosen)	Free — self-hosted	Runs locally, documents never leave server	~5ms per batch	Top 5 on MTEB benchmark
OpenAI ada-002	$0.0001/1K tokens	Documents sent to OpenAI API	~200ms network call	Good but not top-ranked on MTEB
OpenAI text-embedding-3-small	$0.00002/1K tokens	Documents sent to OpenAI API	~200ms network call	Better than ada-002

Using a local embedding model means: (1) documents never leave the server — important for confidential business data, (2) zero per-token cost regardless of how many PDFs users upload, (3) no network dependency for a core pipeline step.

5. LLM INTEGRATION & PROMPT ENGINEERING

Q14. Why did you choose Groq over OpenAI, Anthropic, or running a local model?

LLM Provider	Speed	Cost	Quality	Why Not Chosen
Groq (chosen)	Ultra-fast (~200 tok/s)	Competitive pricing	OpenAI-equivalent models	— Perfect balance
OpenAI direct	Moderate (~40 tok/s)	Higher	Excellent	Slower for a real-time chat app
Anthropic Claude	Moderate	Higher	Excellent reasoning	Slower for this use case
Local (Ollama)	Depends on GPU	Free	Lower for small models	Requires GPU server, ops overhead

Groq's key advantage: Groq uses custom LPU (Language Processing Unit) hardware that runs inference dramatically faster than GPU-based providers. For a chat application where users expect real-time responses, latency matters as much as quality. Groq serves OpenAI-compatible models at OpenAI-grade quality with sub-second response times.

Q15. Why temperature=0 for all your agents? When would you increase it?

Answer: All agents in this system are performing **factual business analysis**, not creative generation. Temperature=0 makes the LLM deterministic — given the same input, it always produces the same output. This is critical for trust and debuggability in a business tool.

High temperature introduces randomness — an analyst agent might give different answers to the same question on two runs. In a business context, that's a bug, not a feature.

Temperature	When to Use	In This Project
0.0 (chosen)	Factual tasks: analysis, extraction, classification, lookup	All 8 agents — always want the same answer for the same query
0.1-0.3	Slight variation acceptable: summaries, query expansion	Query expansion agent uses 0.3 for varied query variations
0.7-1.0	Creative tasks: writing, brainstorming, story generation	Not used — this is a business tool, not a creative tool

Q16. Explain your prompt engineering approach. How did you structure prompts for each agent?

Answer: Each agent prompt follows a structured format: **Role → Context → Task → Output Format**

Example (Analyst Agent): *"You are a senior business analyst. Based on the context below, analyze the query thoroughly... Provide: 1. Root cause analysis, 2. Key patterns, 3. Supporting data points, 4. Risks"*

Prompt Component	Purpose	Example

Role assignment	Sets model persona and tone	"You are a senior business analyst"
Context injection	Grounds the model in retrieved document content	Context from documents: {context}
Task definition	Specific instructions for this agent	"Analyze root causes, identify patterns"
Output format	Structures the response for downstream agents	"Provide: 1. Root cause, 2. Key patterns"
Constraint rules	Prevents hallucination and off-topic answers	"Be specific and data-backed", "Do not use HTML tags"

Q17. How does streaming work in your system? Why implement it?

Answer: For the 'lookup' intent path, the response agent generates the answer using an LLM and streams it token-by-token back to the frontend using FastAPI's **StreamingResponse** with an async generator.

Why streaming: Without streaming, the user sees nothing for 2-5 seconds, then the full response appears. With streaming, the first tokens appear in ~200ms, making the experience feel responsive — like the AI is 'typing'.

Technical implementation: The backend yields each token as a chunk via async generator. The frontend receives these chunks via the Fetch API's ReadableStream and appends them to the UI progressively.

□ PROS / WHY WE CHOSE THIS	□ CONS / TRADE-OFFS
Dramatically better UX — users see progress immediately	More complex frontend code (ReadableStream parsing)
Reduces perceived latency even if total time is the same	Can't post-process the full response before showing it
Allows users to start reading while generation continues	Not used for all paths — analysis/dashboard return full structured data
Industry standard for LLM chat interfaces	

6. FASTAPI & BACKEND ARCHITECTURE

Q18. Why FastAPI over Flask or Django for this project?

Framework	Type	Async	Auto Docs	Why Not Chosen
FastAPI (chosen)	Micro + modern	Native async	Auto OpenAPI/Swagger	— Best fit for AI API service
Flask	Micro	Via extensions	Manual	No native async, no auto-validation, slower
Django	Full-stack MVC	Via Channels	Manual	Too heavy — includes ORM, templates, admin we don't need
aiohttp	Low-level async	Native async	None	Too low-level, no validation, no auto docs

Key FastAPI advantages used in this project:

- Native async/await — critical for streaming responses and concurrent PDF processing
- Pydantic validation — request bodies are automatically validated against schema

- Auto OpenAPI docs — the /docs endpoint is generated for free
- Type hints throughout — TypeScript-level type safety in Python
- UploadFile built-in — multi-file upload with streaming support

Q19. Why did you implement per-user document isolation? How does it work technically?

Answer: Document isolation ensures that User A cannot retrieve content from User B's uploaded PDFs. This is a fundamental privacy and security requirement for any multi-tenant document application.

Implementation: When a user uploads PDFs, the backend receives their user_id from the Authorization header (verified via Supabase). All FAISS indexes are stored in *faiss_indexes/{user_id}/* — a separate directory per user. Every retrieval call passes the user_id and only opens that user's index.

This means: (1) storage is naturally partitioned, (2) one user's query never touches another's FAISS index, (3) users can add/delete their own documents without affecting others.

Q20. How did you implement Prometheus monitoring? What metrics do you track and why?

Answer: I used **prometheus-fastapi-instrumentator** which auto-instruments all FastAPI endpoints (request count, latency histograms per endpoint). On top of that, I added custom metrics:

Metric	Type	What It Tells You
llm_requests_total	Counter	How many LLM calls have been made — helps track usage and billing
llm_errors_total	Counter	How many LLM calls failed — tracks reliability and API issues
llm_response_seconds	Histogram	Distribution of LLM latency — identifies slow queries

Why Prometheus over alternatives: Prometheus + Grafana is the de-facto standard for service monitoring in containerized environments. Cloud Run exposes a /metrics endpoint that can be scraped. Alternative: Cloud Monitoring (GCP native) — would work too but creates vendor lock-in and is harder to run locally.

7. FRONTEND ARCHITECTURE

Q21. Why Next.js over a plain React SPA? Why not Vue or Svelte?

Framework	Rendering	Why/Why Not
Next.js 16 (chosen)	SSR + SSG + CSR	App Router, built-in API routes, Vercel optimized, TypeScript first-class, massive ecosystem
Create React App	CSR only	Deprecated, no SSR, slower initial load, no built-in routing
Vite + React	CSR only	Fast dev, but no SSR, need separate routing library
Vue + Nuxt	SSR capable	Smaller ecosystem, less LangChain/AI community examples
Svelte + SvelteKit	SSR + CSR	Great performance but smallest ecosystem — fewer UI libraries

Key reasons: (1) App Router layout system lets me have separate (auth) and (dashboard) route groups with different layouts. (2) API routes (route.ts) handle the Supabase OAuth callback server-side. (3) Vercel deployment is first-class — zero config CI/CD for the frontend. (4) TypeScript support is excellent.

Q22. Why Supabase for auth? Why not Firebase, Auth0, or building your own?

Auth Provider	Pros	Cons / Why Not Chosen
Supabase (chosen)	Open source, Postgres DB included, SSR support (@supabase/ssr), generous free tier, Google OAuth built-in	Smaller community than Firebase
Firebase Auth	Very mature, Google-backed, huge ecosystem	Vendor lock-in, NoSQL only (Firestore), complex pricing at scale
Auth0	Feature-rich, enterprise-grade	Expensive at scale, adds another external dependency
Custom JWT	Full control	Security responsibility, significant development time

The key advantage: Supabase SSR (@supabase/ssr) handles session management server-side in Next.js middleware — meaning protected routes are enforced before the page even renders, not after JavaScript loads. This prevents flash-of-unauthenticated-content and is more secure than client-side auth guards.

Q23. Explain the DashboardRenderer component — how does it work?

Answer: DashboardRenderer is a React component that takes **LLM-generated React code as a string** and renders it dynamically in the browser.

Flow: (1) User asks for a dashboard. (2) DataExtractor agent extracts structured JSON from document context. (3) Dashboard agent uses the JSON data to generate a complete React component string (function Dashboard() {...}) with Chart.js charts. (4) The response agent packages it as JSON. (5) Frontend's DashboardRenderer dynamically evaluates and renders this React function.

This is **code generation as a feature** — the LLM doesn't just answer text questions, it generates executable UI components that visualize the data. Chart.js is available as window.Chart so the generated code can use it without import statements.

Interview Tip: Be ready to discuss the security implications — dynamically evaluating LLM-generated code is a risk. Mitigations: the code runs in the frontend (no server access), sandbox it in an iframe for production, validate the response JSON schema before rendering.

8. DOCKER & CLOUD DEPLOYMENT

Q24. Walk me through your Dockerfile. What optimizations did you make?

Answer: The Dockerfile follows production best practices with several deliberate optimizations:

Optimization	How Implemented	Why It Matters
Slim base image	python:3.11-slim not python:3.11	Reduces image size by ~600MB — faster pull, smaller attack surface

Layer caching	Copy pyproject.toml/uv.lock BEFORE code	Dependency layer only rebuilds when deps change, not on every code change
uv package manager	RUN pip install uv then uv sync	uv is 10-100x faster than pip — much faster CI/CD builds
No dev dependencies	--no-dev flag in uv sync	Excludes pytest, pytest-mock etc — smaller image in production
Precise COPY	Copy only app/, agents/, rag/, workflows/	Excludes .venv, tests, htmlcov, data from image
PORT 8080	EXPOSE 8080 + uvicorn --port 8080	Cloud Run expects port 8080 by default

Q25. Why Google Cloud Run over AWS EC2, AWS ECS, or Kubernetes?

Platform	Type	Why Not / When Better
Cloud Run (chosen)	Serverless containers	Zero server management, scales to zero (no cost when idle), pay-per-request
AWS EC2	VMs	Always-on cost even when idle, requires manual scaling config, more ops overhead
AWS ECS (Fargate)	Container service	Similar to Cloud Run but higher config complexity, no scale-to-zero by default
Kubernetes (GKE/EKS)	Container orchestration	Overkill for single-service project — massive ops overhead, expensive
App Engine	PaaS	Less control, harder to containerize exactly as needed

Cloud Run ideal for this project because: (1) The app has bursty traffic — students/users query occasionally, not constantly. Scale-to-zero means $0 during idle periods. (2) Deployment is one gcloud command. (3) 2Gi memory handles the sentence transformer models. (4) 300-second timeout accommodates long RAG + LLM calls.

9. CI/CD & TESTING STRATEGY

Q26. Explain your CI/CD pipeline. What happens on every push to main?

Answer: The GitHub Actions workflow (ci-cd.yml) has two jobs:

Job 1 — Test (runs on every push AND pull request):

- Checks out code on ubuntu-latest runner
- Installs uv with dependency caching (cache key = uv.lock hash)
- Runs: uv run pytest --cov --cov-report=term-missing
- Tests ALL agents, RAG components, and API endpoints

Job 2 — Deploy (only on push to main, only if tests pass):

- Authenticates to GCP using stored GCP_SA_KEY secret
- Creates env.yaml from GitHub Secrets (GROQ_API_KEY, SUPABASE_URL, etc.)
- Runs: gcloud run deploy ai-business-copilot --source . (builds Docker image in Cloud Build)
- Reports deployed URL

Key Design: PRs only run tests, not deploy. This means I can validate changes safely before they hit production. Main branch = always deployed = always working.

Q27. How did you test agents that depend on LLM calls? How do you mock external dependencies?

Answer: Testing LLM-dependent code requires mocking — you can't call the real Groq API in CI (cost, flakiness, rate limits). I used **pytest-mock** to replace the ChatGroq object with a mock that returns controlled responses.

Pattern used:

- conftest.py defines a mock_llm fixture using mocker.patch("agents.analyst_agent.llm")
- mock_llm.invoke.return_value.content = 'Expected test response'
- Tests then call the agent with known input state and assert the output state is correct

For the FAISS/RAG tests: I mock *load_vector_store_for_user()* to return a mock store with a controlled *similarity_search()* return value. This tests the retrieval logic without needing actual embeddings.

PROS / WHY WE CHOSE THIS	CONS / TRADE-OFFS
Tests run fast — no actual API calls	Mocks must be updated when agent interfaces change
Tests are deterministic — no LLM randomness	Don't test the actual LLM response quality
Can test edge cases easily (e.g. empty context, NO_DOCUMENTS)	Risk of tests passing but real LLM producing different output format
No API key required in CI — just a dummy GROQ_API_KEY=test	

Q28. Why uv instead of pip? What is uv and why is it faster?

Answer: uv is a Rust-based Python package installer and resolver created by Astral (the team behind Ruff). It replaces pip, pip-tools, virtualenv, and pyenv in one tool.

Aspect	pip	uv
Speed	Baseline (slow resolver)	10-100x faster — Rust implementation with parallel downloads
Lock file	requirements.txt (no hashes by default)	uv.lock — cryptographically verified, reproducible builds
Virtual env	Separate tool (virtualenv/venv)	Built-in: uv venv creates it
Reproducibility	requirements.txt can drift	uv sync --frozen guarantees exact versions from lock file
Docker caching	Layer invalidates on any dep change	Same behavior, but faster when cache misses

In CI/CD: a cold pip install of all dependencies might take 2-3 minutes. uv does the same in 15-30 seconds. Over dozens of CI runs per day, this adds up significantly.

10. DEEP-DIVE TECHNICAL QUESTIONS

Q29. How would you scale this system if user volume grew 100x?

Answer: Several components would need to change at 100x scale:

Component	Current (Works for ~100 users)	At Scale (10,000+ users)
Vector Store	FAISS on local disk per user	Migrate to Qdrant or Pinecone with multi-tenant collections
Document Storage	Temp files + FAISS on container disk	Store PDFs in Cloud Storage (GCS), index in managed vector DB
LLM Calls	Sequential per request	Add request queuing (Cloud Tasks), async processing
Backend	Single Cloud Run instance	Cloud Run auto-scales — already handled, just increase max instances
Embeddings	CPU-based sentence-transformers	GPU instance or use OpenAI/Cohere embeddings API
Auth & Sessions	Supabase (managed)	Already scalable — Supabase handles this

Q30. What would you do differently if you rebuilt this project today?

Answer: Three things I would change:

- [object Object]Right now I have Prometheus for latency metrics but no visibility into what the LLM is generating for each query. LangSmith would give me a trace of every agent call, making debugging much easier.
- [object Object]Currently, PDF ingestion blocks the API response. I would move it to a background task (FastAPI BackgroundTasks or a queue like Cloud Tasks) and use a WebSocket or polling endpoint to notify the frontend when indexing is complete.
- [object Object]I have no way to measure if my RAG answers are correct. I would add RAGAS (RAG Assessment) to measure faithfulness, context relevancy, and answer relevancy on a test set of known Q&A pairs from sample documents.

Q31. Explain the difference between your Analyst agent and Strategy agent. Why are they separate?

Answer: This is a classic **separation of concerns** in multi-agent design. Each agent has a single, well-defined responsibility:

Agent	Job	Input	Output
Analyst Agent	Understand WHAT happened and WHY	Query + retrieved document context	Root cause analysis, patterns, data points
Strategy Agent	Recommend WHAT TO DO	The analyst's findings	Prioritized recommendations, short/long-term actions

Why separate: (1) **Focused prompts are better** — an analyst prompt that says 'find root causes' produces better analysis than a single prompt asked to do both. (2) **Modularity** — I can swap the strategy agent for a different LLM or logic without affecting the analyst. (3) **Testability** — I can unit test each agent independently.

Q32. What security considerations did you implement? What would you add?

Already implemented:

- Supabase JWT authentication on every API request — user_id extracted from verified token
- Per-user document isolation — FAISS indexes are namespaced by user_id
- CORS middleware — only allows requests from known frontend origins
- Secrets in environment variables — never hardcoded, injected at runtime via Cloud Run
- GitHub Secrets — GROQ_API_KEY, GCP keys never in code, managed by Actions secrets
- .dockerignore / .gcloudignore — .env files excluded from Docker image and Cloud Build

Would add in production:

- Rate limiting — prevent API abuse per user (e.g., slowapi middleware)
- Input sanitization — validate PDF files (MIME type check, file size limit, virus scan)
- DashboardRenderer sandboxing — render LLM-generated code in a sandboxed iframe
- Audit logging — log which user queried what and when (for compliance)

11. INTERVIEW TIPS & CLOSING

How to structure your project walkthrough (2-3 min pitch):

- Start with the problem: 'Business users waste hours manually reading reports. I built an AI assistant that lets them ask natural language questions about their own documents.'
- Architecture in one sentence: 'FastAPI backend, LangGraph multi-agent system with 4 paths, advanced RAG pipeline, deployed on Cloud Run with a Next.js frontend.'
- Highlight the most impressive technical decisions: Hybrid search, cross-encoder reranking, per-user isolation, AI-generated dashboards, full CI/CD pipeline
- Close with impact: 'A user can go from uploading a PDF to getting a KPI dashboard in under 30 seconds.'

Questions to expect:

- 'How is this different from just using ChatGPT with a file?' → Answer: Per-user isolation, hybrid search, reranking, conditional agent routing — all custom-built
- 'How do you ensure the answers are accurate?' → Answer: Retrieved chunks are injected as context (not memory), reranking improves relevance, temperature=0 reduces hallucination
- 'What's your biggest technical challenge?' → Answer: The DashboardRenderer — generating React code from LLM and rendering it safely in the browser was non-trivial

Remember: Always frame decisions as trade-offs, not right/wrong. 'I chose FAISS because at my current scale the benefits of simplicity and zero cost outweigh the limitations. At scale I would migrate to Qdrant.' This shows maturity.